

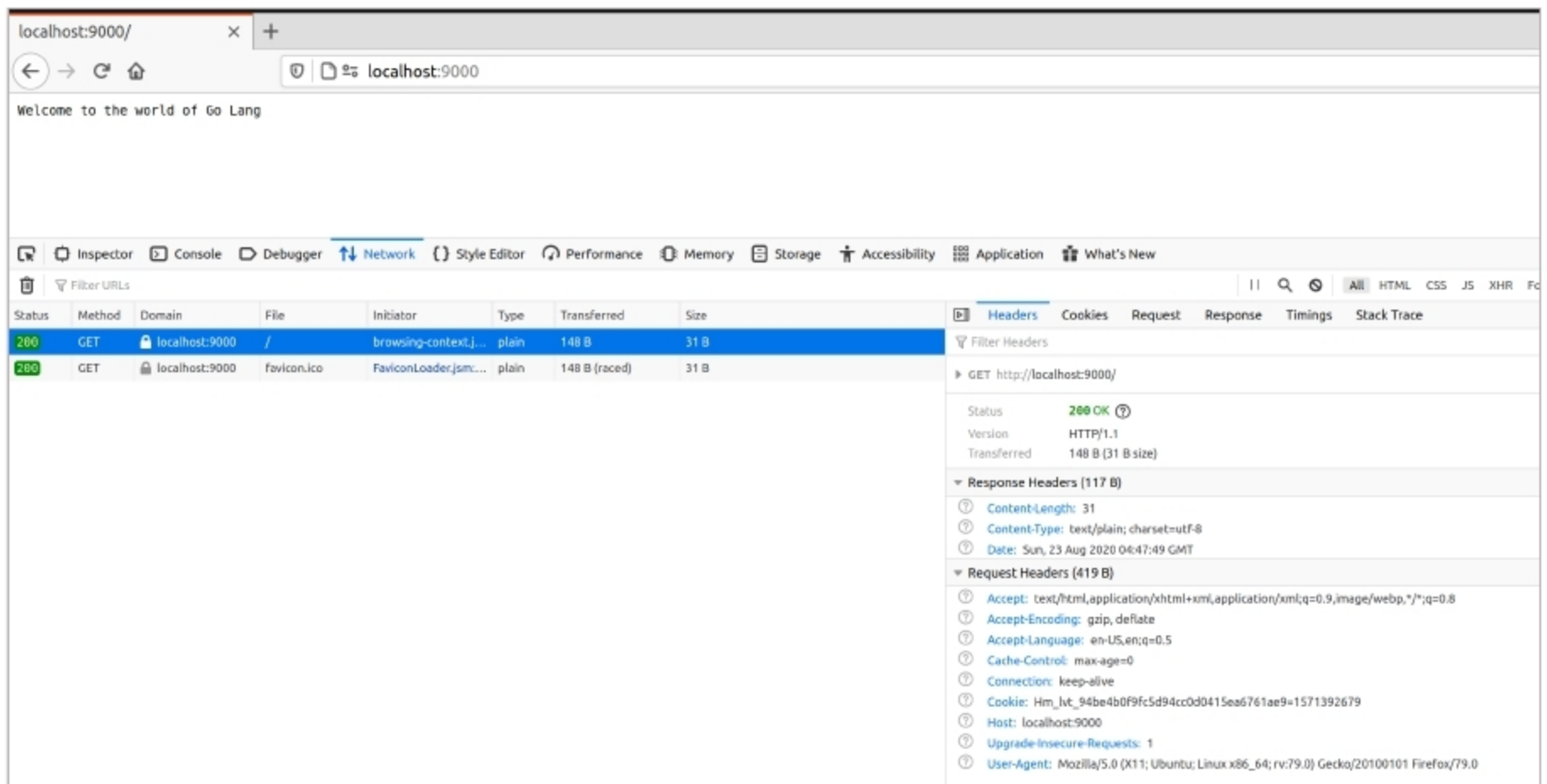


Figure 2: Browser

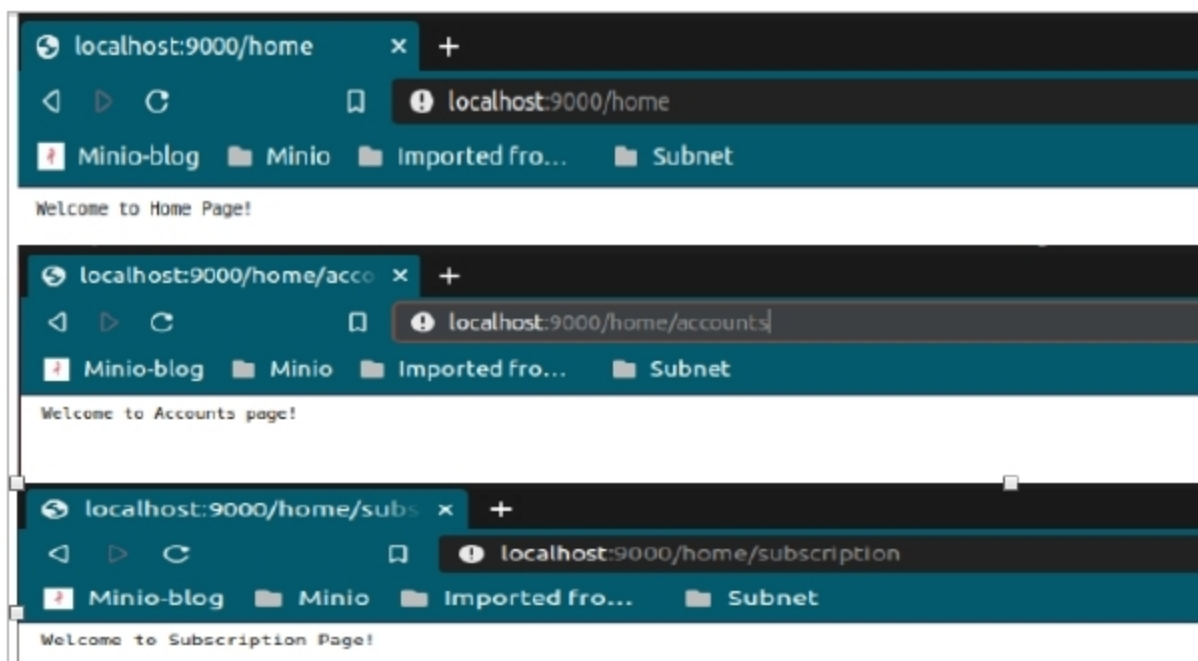


Figure 3: Mux

The code below depicts different outputs for the URLs `/home`, `/home/accounts`, `/home/subscription`:

```
package main
```

```
import (
    "fmt"
    "net/http"
)
```

```
func main() {
    // Create a new ServerMux
    mux := http.NewServeMux()
```

```
// handle `` route
mux.HandleFunc("/home", func(res http.
ResponseWriter, req *http.Request) {
    fmt.Fprint(res, " Welcome to Home
Page!")
})
```

```
// handle `Accounts` route
mux.HandleFunc("/home/accounts",
func(res http.ResponseWriter, req
```

```
*http.Request) {
    fmt.Fprint(res, " Welcome to Accounts
page!")
})
```

```
mux.HandleFunc("/home/subscription",
func(res http.ResponseWriter, req
*http.Request) {
    fmt.Fprint(res, " Welcome to
Subscription Page!")
})
```

```
// Listen and serve using ServerMux
http.ListenAndServe(":9000", mux)
```

```
}
```

This article has covered the basics of setting a server in Go language. For further details, one can go through the documents at <https://golang.org/>.

In order to learn Golang one can visit <https://tour.golang.org/welcome/1> and work on a hands-on session at <https://play.golang.org/>. 

 By: Tanwi Shree

The author is an open source software and GNU/Linux advocate, and an avid writer living in Bengaluru. She likes to write about how technology affects people, and vice versa.